
CSE 151B Project Final Report

Yiming Zhao
yiz231@ucsd.edu

Jiatu Li
jil165@ucsd.edu

Jinya Jiang
j9jiang@ucsd.edu

Ziling Gong
zgong@ucsd.edu

1 Task Description and Background

1.1 Deep Learning Task and Its Importance

To improve the efficiency of electronic taxi dispatching systems, it is important to be able to predict how long a driver will have his taxi occupied. If a dispatcher knew approximately when a taxi driver would be ending his current ride, they would be better able to identify which driver to assign to each pickup request.

In this project, the task at hand is to leverage historical taxi ride data to predict the duration of future taxi rides in Porto, Portugal. This predictive task carries significant importance as it aligns with our ultimate objectives of optimizing taxi operations and improving transportation by providing accurate estimates of trip duration.

The training dataset comprises 1,710,670 entries, each with 9 relevant features — `Call_Type`, `Origin_Call`, `Taxi_ID`, `Timestamp`, `Day_Type`, `Missing_Data`, and `Polyline`. These features represent the type of service request, the identifier of the customer, the taxi identifier, the timestamp of the initial call, the categorization of the day, the presence of missing data, and a list of GPS coordinates forming the polyline of the trip, respectively. The objective of this prediction task is to estimate the trip duration, which is given by the number of intervals in `Polyline`, each interval amounting to 15 seconds. This can be mathematically represented as:

$$y = (n - 1) \times 15 \quad (1)$$

where n is the number of polyline points.

The test dataset consists of 320 entries, each with 8 features. These features are identical to those in the training dataset, excluding `Polyline`. The task is to build a deep learning model that uses features such as `Call_Type`, `Origin_Call`, `Taxi_ID`, `Timestamp`, and `Day_Type` to estimate the trip duration,

$$\hat{y} = f(X|\hat{\theta}) \quad (2)$$

where X is the input feature, $\hat{\theta}$ is the learnt parameters.

Additionally, a supplementary meta dataset is provided, which includes information about taxi stands such as `stand_id`, `Descricao`, `Longitude`, and `Latitude`. This dataset is intended to provide additional context when a trip is directly requested from a taxi driver at a specific stand.

Successfully addressing the task of predicting taxi trip duration holds substantial implications for both individuals and society at large. This achievement can significantly improve commuting experiences

by enabling commuters to better plan their journeys, optimize their schedules, minimize waiting times, and avoid unnecessary delays. Moreover, through the analysis of historical taxi trip data and the accurate prediction of travel times, traffic management authorities can gain valuable insights into traffic patterns and congestion levels. This knowledge empowers them to identify traffic bottlenecks, optimize road infrastructure, and implement targeted measures to alleviate congestion. Overall, the proficient prediction of taxi trip durations enhances transportation efficiency, reduces congestion, and contributes to the development of more streamlined and sustainable urban environments.

1.2 Mathematical Formulation of the Prediction Task

Consider the input vector $\mathbf{X}^{(n)} = [x_1, x_2, \dots, x_d]$, where each x_i represents an input feature (such as Call_Type, Taxi_ID, Day_Type, etc) of the n th instance in the dataset. The output $\hat{y}^{(n)}$ denotes the predicted duration of the trip (see equation 2). Given a continuous quantitative output, we utilize the Root Mean Square Error (*RMSE*) as our loss function, which is given by:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2} \quad (3)$$

where y_i is the observed trip duration (see equation 1), $\hat{y}_i$ is the predicted trip duration (see equation 2), and n is the total number of observations in the dataset.

In this project, the optimization of our predictive model is pursued through the minimization of the root mean square error (*RMSE*) between the actual travel time and the predicted travel time. The objective is to achieve a lower *RMSE* in the validation/test phase, indicating a better model performance and improved accuracy of predictions.

The key requirement for employing a regression model is that the output of the task should represent a continuous quantitative value. Indeed, the general mathematical abstraction of the model holds relevance across various regression tasks, for example:

- Finance. In the context of predicting future stock prices, a regression model can be trained using historical stock data and relevant features to forecast the future price movements.
- Health Care. A regression model can be utilized to analyze patient data and factors such as medical history, vital signs, and demographics to estimate the duration of a patient’s stay in the ICU.

By leveraging appropriate regression techniques, such as linear regression, decision trees, or neural networks, we can effectively model and predict continuous outcomes for a wide range of tasks. The flexibility of regression allows for the application of this methodology in diverse domains, enabling accurate forecasts and valuable insights.

2 Exploratory Data Analysis

2.1 Dataset Description and Analysis

The training dataset comprises 1,710,670 rows and 9 columns of Porto taxi rides, while the test dataset consists of 320 rows and 8 columns. The output (predicted trip time) will be written in the submission file, which has 320 rows and 2 columns (Trip_ID and predicted Travel_Time). The metadata has 63 rows and 4 columns.

In the raw data, input has a dimension of (1710670, 9), while sample submission has a dimension of (320, 2).

Data wrangling was conducted at first. The training set and the testing set have a large amount of NA value in Origin_Call and Origin_Stand, which are filled with the mode grouped by Taxi_ID for

Origin_Call and Origin_Stand. Timestamp is transformed to `pandas.datetime` so it is easy to extract time features that can be trained by some specific models like XGBoosting.

Furthermore, the longitude and latitude of the origin stand are retrieved after merging the training set with metadata set on Origin_Stand. The distribution of trip position is displayed by a heatmap in Figure 1, in which some outliers that have extremely long distances (travel time $\geq 10,000$ seconds) from Porto are truncated. After that, a new feature is added by calculating the Haversine distance from the city center to the stand location using the formula:

$$\text{haversine}(\theta) = \sin^2 \frac{\theta}{2} \quad (4)$$

$$d = 2r \cdot \arcsin \left(\sqrt{\text{haversine}^2(\phi_2 - \phi_1) + \cos(\phi_1) \cdot \cos(\phi_2) \cdot \text{haversine}^2(\lambda_2 - \lambda_1)} \right) \quad (5)$$

where, ϕ_1 and λ_1 are the latitude and longitude of the city center Averio ($-8.610989, 41.148932$), ϕ_2 and λ_2 are the latitude and longitude of the taxi stand, r is the radius of the Earth (6371 km). This feature leverages the information from origin stand, which helps to remove the outliers at distant.

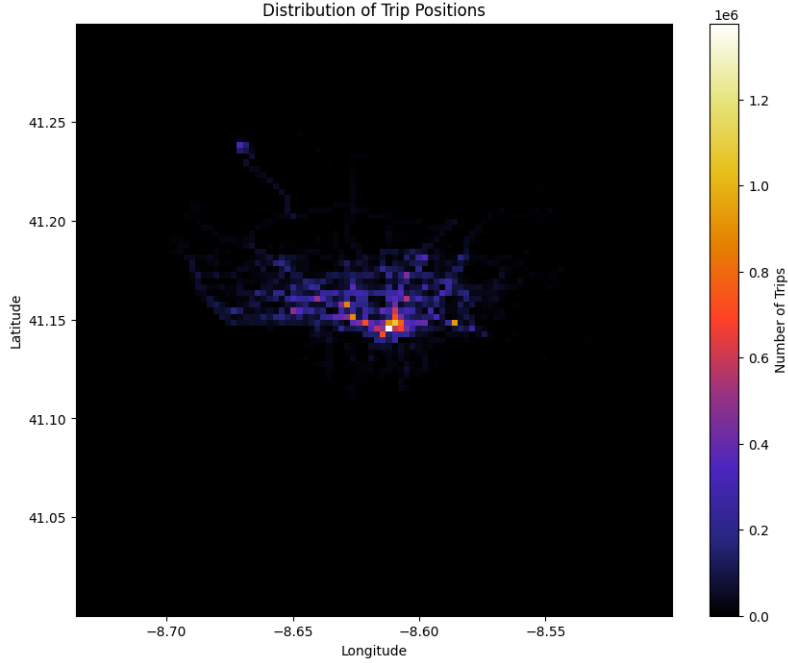


Figure 1: Distribution of Trip Positions

Moreover, analysis on the travel time is conducted on the training dataset, and the distribution of trip time is shown in Figure 2. The distribution of trip times limits ranges within 1 hour as there exists extremely large travel times. It is found that the distribution is skewed to the right and the majority of trip times are centered around 10 minutes.

There are 1700576 trips with travel time ≤ 1 hour.

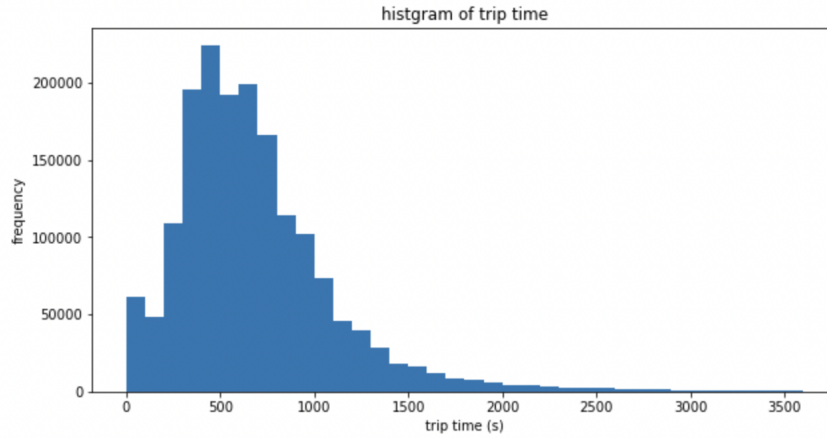


Figure 2: Distribution of Trip Time

More analysis is about the train-test balance. The distribution of `Call_Type` in training set and testing set in Figure 3 demonstrate if there exists an imbalanced type. The figure shows that `Call_Type` is a balanced data. Moreover, it reveals the disadvantages of using the Haversine distance from origin stand to the city center, which is calculated based on the call type B, that accounts for 50% of training set and 40% of test set.

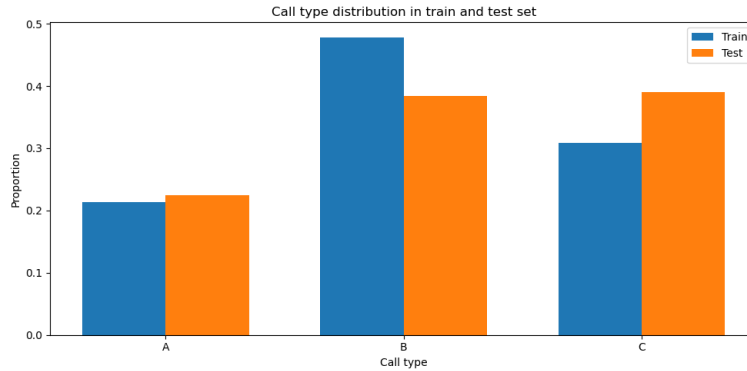


Figure 3: Distribution of Call Type

The training set has 134590 missing values in `Origin_Call` and 904091 in `Origin_Stand`. The testing set has 248 in `Origin_Call` and 197 in `Origin_Stand`. Both of them are filled by the mode of same `TAXI_ID`. If there is no available mode, the mean for whole dataset will be used to fill NA.

The chosen dimensions for input are shown in Figure 4. The output dimension will be only predicted travel time. The `TRIP_ID` will be concatenated. Sampled data for the final features are displayed below in Figure 4.

2.2 Data Processing and Feature Engineering

As described in section 2.1, NA is filled in `Origin_Call` and `Origin_Stand` with the mode, from which Distance from the city center to the `Origin_Stand` was calculated. To consider the periodicity of the interpretability features, we first considered to one-hot encode the features Month, Hour, and DayOfWeek. A heatmap (Figure 5) of `TRAVEL_TIME` on the relationship between the day of week and the month displays that in the months except for July and August, average travel

Index	ORIGIN_CALL	ORIGIN_STAND	TAXI_ID	MISSING_DATA	Latitude	Longitude
731413	13775	56	35	0	41.162879	-8.591645
1276762	2002	57	325	0	41.145719	-8.610707
1044364	13243	15	399	0	41.148628	-8.585877
86862	2002	23	45	0	41.146016	-8.612575
1581577	2002	15	238	0	41.148628	-8.585877
Index	DISTANCE	SECOND	MINUTE	HOURL	DAY	MONTH
731413	2.242251	-0.994522	0.104528	0.258819	0.866025	-2.45E-16
1276762	0.35807	-0.104528	0.406737	-0.707107	0.743145	8.66E-01
1044364	2.102938	-0.809017	-0.866025	0.5	0.406737	8.66E-01
86862	0.350394	-0.5	-0.669131	0.866025	-0.743145	-5.00E-01
1581577	2.102938	0.866025	-0.587785	-0.965926	0.866025	1.22E-16
Index	DAYOFWEEK	CALL_TYPE_A	CALL_TYPE_B	CALL_TYPE_C	DAY_TYPE_A	
731413	0.433884	1	0	0	1	
1276762	-0.433884	0	0	1	1	
1044364	0.433884	1	0	0	1	
86862	-0.433884	0	1	0	1	
1581577	0.433884	0	1	0	1	

Figure 4: Sample Training Data for Final Model

time is high from Monday to Friday in general. Figure 6 showed that average travel time is high from 7AM to 11PM and 3PM to 6PM. Thus, we categorized interpretability features into HOUR: {7-11pm, 12-2pm, 3-6pm, 7-12pm, 12am-6am}, WEEK: {Weekday, Weekend}, MONTH: {1-6, 7-9, 10-12}. More experiments with this input data will be discussed in section 4.

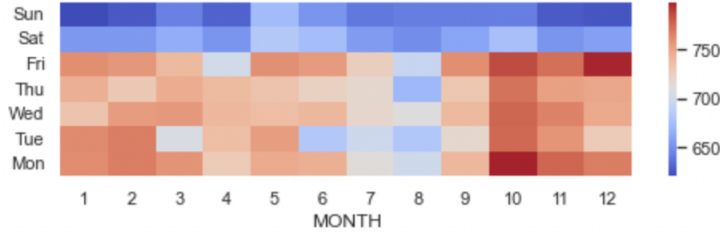


Figure 5: Long Travel_Time on Mon to Fri

For TAXI_ID, we consider it as quantitative values instead of categorical ones. TAXI_ID was re-indexed to 0-445 to get rid of prefix (20000-index). Although one-hot encoding taxi_id seems more logical, our experiments revealed that its quantitative presence reduced *RMSE* and improved model performance. The reason behind it is hard to interpret but we guessed it was because our data is noisy and the large values of the id added variance to the final output.

After revealing the impacts from *datetime*, we use trigonometric functions to encode them in our final model instead of OHE which is memory-consuming. For example, Hour column is transformed by

$$hour \leftarrow \sin(2\pi \times \frac{hour}{24}) \quad (6)$$

Training set and validation set are divided using the ratio 4:1 by the Scikit-Learn package. The training size is 1367722 and validation size is 327411. For estimators which are not tree-based, we standardized our input by using `sklearn.StandardScaler`. It eliminates scale differences between features, improving model convergence and enhancing performance. One-hot encoded Call_Type feature was employed in our final model. Since call types A and C missed Origin_Stand information, we only recorded the Distance feature for type B rows. Though we thought to train different call type individually, the implementation was not plausible. The most importance part of

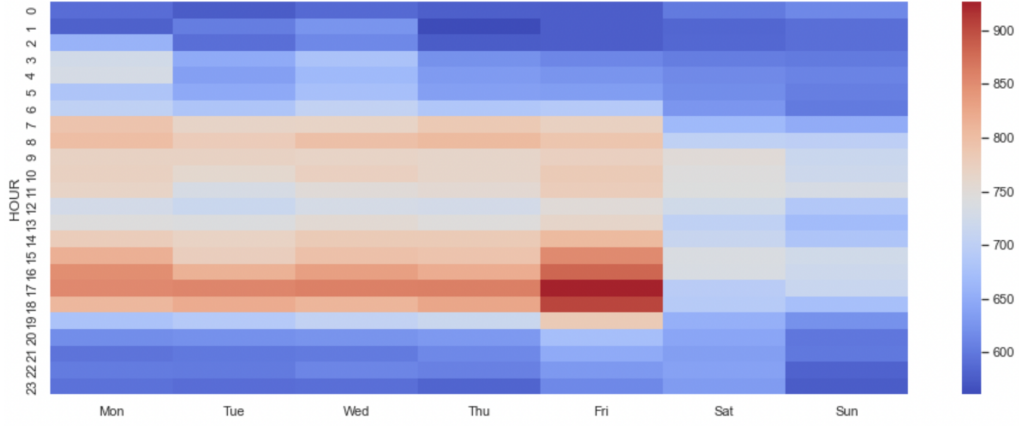


Figure 6: Long Travel_Time in the afternoon

Call_Type is mentioned in Section 2.1, where the Origin_Stand helps find the starting point of travel.

3 Deep Learning Model

3.1 Deep Learning Pipeline

Simple Multi-layer Perceptron (MLP) model was implemented with the structure shown in Figure 7. The input features are in Figure 4. To be specific, TAXI_ID is the one after reindexing to 0-445, SECOND/MINUTE/HOUR/DAY/MONTH/DAYOFWEEK are the ones after sine or one-hot encoding. The output is predicted travel time. The loss function is *MSE* loss since our final evaluation is based on *RMSE*.

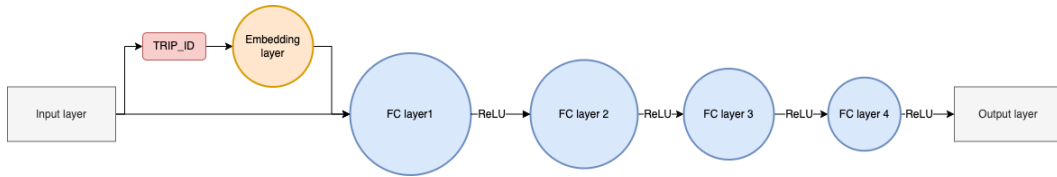


Figure 7: MLP structure

The training loss and validation loss are in Figure 8. After that, 10 travels with the largest value $abs(\hat{y} - y)$ are chosen to plot their travel maps, see Figure 9. The figure shows that most of them have departures/destinations far away from the city.

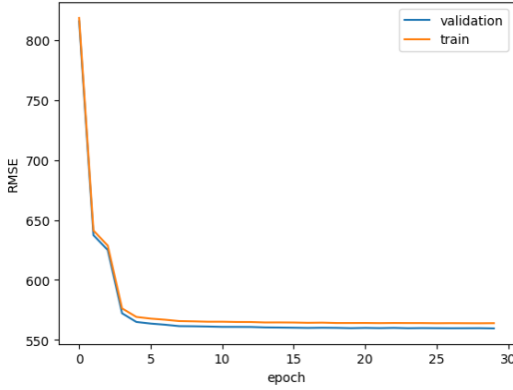


Figure 8: MLP training loss and validation loss

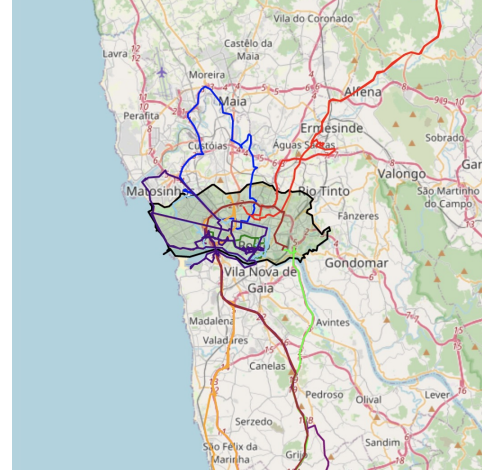


Figure 9: 10 travels with the largest loss

Figure 10: Result of MLP

However, MLP is not our final choice. We chose to use an ensemble of XGBoosting Regressors as our final model demonstrated in Section 3.2 and Figure 11.

3.2 Experiments and Attempted Models

Not only deep learning models are selected, but simple model like logistic regression, ensemble learning like XGBoosting Regressor, or even our models with our own design structures are tested. There are several models, namely:

- **Logistic Regression:** The model was trained on a subset of 320,000 samples drawn from the training set.
- **XGBoosting Regressor:** A single regressor using the entire training set. The hyperparameters were set as follows: *learning_rate*=0.15, *n_estimators*=800, *max_depth*=7, *min_child_weight*=1, *gamma*=0.0, *colsample_bytree*=0.7, and *subsample*=0.7. Chen and Guestrin [2016]
- **Multi-layer Perceptron:** This model consisted of 4 fully connected layers with ReLU activation functions. The parameters included: *epoch*=30, *batch_size*=256, *learning_rate*=0.01, and used the Adam optimizer.
- **Multi-layer Perceptron with Embedding Layer:** This model was similar to the aforementioned MLP but also included an embedding layer. The parameters included: *epoch*=30, *embedding_size*=22, *batch_size*=256, *learning_rate*=0.01, and used the Adam optimizer.
- **Ensemble of XGBoosting Regressors:** For this method, XGBoosting regressor with the best validation error was chosen and updated the potential extremas' travel time. See details in later part.

The ensembles of XGBoosting regressors is chosen as the final model. The architecture is shown in Figure 11. Most regressors have the limit that they can not predict any travel time larger than 4000 or higher in most of time. However, it is those extremas that greatly lead to a highly *RMSE* value. To demonstrate this, we use python to generate 320 fake ground truth for travel time, in which $y_i \sim N(660, 784)$ and $y_i \geq 0$, where 660 is the average travel time in training set and 784 is the public score for sample submission. Suppose we use sample submission as the prediction, the *RMSE* is around 600. However, if there are extremas like the training set has, the *RMSE* is around 900. The difference between these two *RMSEs* in Figure 12 is similar to what can possibly occur in the validation error and the Kaggle test error. Therefore, it claims the necessity to manually increase the extremas' predicted time. However, it is hard to classify one TRIP_ID as extrema – a misclassification will lead to a higher *RMSE*. Referring to the general

ensemble learning structure, we came up with an ensemble of XGBoosting regressors. (see Figure 11)

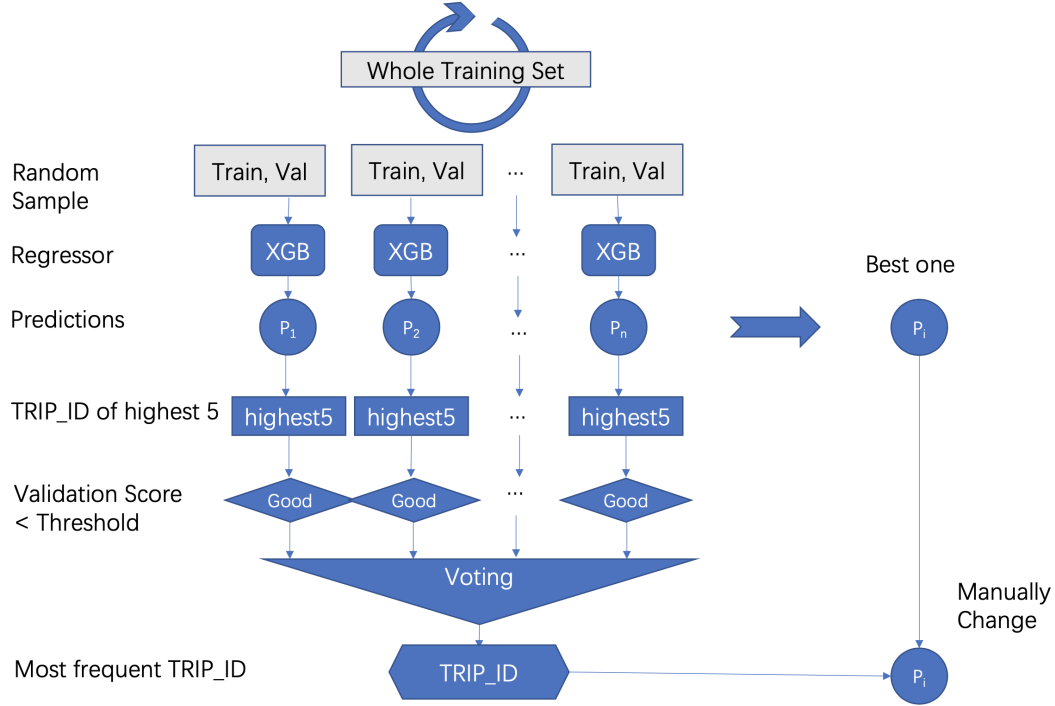


Figure 11: Ensemble of XGBoosting regressors

This model consists of thousands of XGBoosting regressors. Each XGBoosting regressor is trained by 320,000 samples from training set and validated by 3,200 samples from the rest of training set. In each prediction by single model based on test data, there are 5 largest travel time's TRIP_ID. If the validation error of a single model is below the threshold, it is chosen as a good model to participate in voting. By the voting mechanism, our model counts the frequency of TRIP_ID in the chosen models' 5 largest travel time. Finally, we manually increased the travel time based on the frequency of TRIP_ID and its predicted travel time in the best model that has the smallest validation *RMSE*. To be conservative, we just change one of the most frequent to 9000, and change others to 5000. If the frequency is not high enough while its prediction is high, we keep it.

	A	B	C	D
1	Fake GT	Prediction	Fake GT with extremas	Prediction
2	2543.6865	660	9000	660
3	2450.5345	660	7000	660
4	2414.1586	660	5000	660
5	2403.9976	660	5000	660
6	2325.1383	660	4000	660
7	2240.6409	660	2240.640935	660
321	13.846155	660	13.84615542	660
322	RMSE	599.19384	RMSE	897.60631

Figure 12: *RMSE* shift

For the regularization, dropout is chosen in MLP model. However, XGBoosting using an elastic regularization (a combination of L1 and L2) showed no improvement in hyper-parameter tuning.

4 Experiment Design and Results

4.1 Model Designs

The GPUs used for training and testing are: 2 NVIDIA RTX 3090 and 1 NVIDIA RTX 2080Ti, provided by HKU CS GPU Farm – the home university of our team member, Yiming Zhao. Further details about environment and platform is at the github page.

We have a non-deep learning model as our state-of-the-art. To test the performance of deep neural networks, a Multilayer Perceptron model is implemented with an Adam optimizer using epoch 30, batch-size 256, and learning-rate 0.01 which will decay by 0.5 after 50000 iterations. It takes roughly 20 seconds for each epoch and 10 minutes for all epochs.

To test the whether the processing for TRIP_ID will affect the result, another Multi-layer Perceptron model with an additional embedding layer using embedding_size 22 is trained. Other settings are same as the simple one. At last, it has a similar performance as the simple one, which indicate that it is not necessary to embed the TRIP_ID.

The past experience highly motivates the choices and design on models. A simple model (logistic regression) is used for testing the correctness of pipeline (pre-processing, training, predicting, evaluating, submitting).

An ensemble model like random forest or XGBoosting is capable of analysing the importance of feature, and has a not-bad performance usually. The reason for choosing XGBoosting is that it is able to be accelerated by GPU. In this task, the huge training set takes an unpredictable long time to train using random forest or XGBoosting without GPU. Furthermore, an extremely short training enables finer tuning which shows the best effect that this set of features can achieve.

```
# use XGBoosting in GPU mode
import xgboost as xgb
xgbreg = xgb.XGBRegressor(n_jobs=-1, tree_method='gpu_hist', random_state=151)
xgbreg.set_params(learning_rate=0.15, n_estimators=800, max_depth=7, min_child_weight=1, gamma=0.0, colsample_bytree=0.7, subsample=0.7)
xgbreg.fit(X_train, y_train)
y_pred = xgbreg.predict(X_val)
print(' Validation_RMSE:', mean_squared_error(y_val, y_pred)**0.5)
```

Since XGBoosting is a black-box that lacks interpretability, a neural network is necessary. It is chosen to test the potential of Neural Networks. MLP is chosen because typical models like CNN is for graphical data and the test data lacks of time sequence needed by RNN. Therefore, a simple MLP is created for evaluating the performance of the Neural Network.

```
# build a simple MLP model
class MLP(nn.Module):
    def __init__(self):
        super(MLP, self).__init__()
        self.linear1 = nn.Linear(17, 128)
        self.linear2 = nn.Linear(128, 64)
        self.linear3 = nn.Linear(64, 32)
        self.linear4 = nn.Linear(32, 1)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(p=0.2)

    def forward(self, x):
        ...
```

4.2 Model Comparison

In this study, five representative different experiments were conducted, each with unique settings.

Experiments 1, 2, and 3 utilized the features listed in Figure 4, with the interpretability features represented as quantitative values. While the features are same as the ones in figure, the features here were not encoded by sine function like that in the figure.

For Experiment 4, the same set of features from Figure 4 was used, but with the interpretability features one-hot-encoded as categorical variables.

For Experiment 5, the Figure 4 are exactly those used. The interpretability features were encoded by triangular function.

Experiment 1: The XGBRegressor was implemented with the following hyper-parameters: *learning_rate*=0.15, *n_estimators*=800, *max_depth*=7, *min_child_weight*=1, *gamma*=0.0, *colsample_bytree*=0.7, and *subsample*=0.7. Hyper-parameter tuning was conducted using grid search.

Experiment 2: An MLP model with four layers was employed, utilizing linear layer and ReLU activation.

Experiment 3: An MLP model with an embedded TRIP_ID was used. An embedding layer was added before the fully connected layers. The consecutive TRIP_ID values were passed into the embedding layer, and the embedded TRIP_ID was added back to the original features in X_train. The architecture of the MLP is depicted in Figure 7.

Experiment 4: A Random Forest model was implemented, including interpretability features.

Experiment 5: An ensemble of the XGBoosting regressor described in Section 3.2 was employed. 6240 models were trained and validated on different subsets of the training data (with a training time of 3 seconds per model). Models with validation error smaller than 440 were chosen to vote. Post voting, several TRIP_IDs were selected as extremas based on their frequency (as shown in Table 1). The model with the lowest validation error was chosen to adjust some TRIP_ID's travel times according to their frequency. In our two submissions, we both tried to change T208 to 9000, T174 to 5000 and T208 to 5000, T174 to 9000.

Table 1: TRIP_ID frequency

TRIP_ID	T208	T174	T98	T211	T206	T78	T132	T228	T71	...
Frequency	9	9	4	4	4	3	3	3	3	...

The Table 2 shows the experiments performance.

To reduce the time of training:

- Use better GPU like RTX 3090 which supports NVLink to leverage the advantages of multi-GPU calculating.

Table 2: Model comparison

Representative Model	Valid_RMSE	Training size	parameter number	performance	Kaggle Score
Logistic Regression	~700	320000	13338	13 mins	not tested
XGBRegressor (Figure 4)	~530	1367722	~ 204800	7 s	~745
XGBRegressor(Figure 7)	~561	1367722	~ 204800	7 s	~not tested
MLP (Figure 4)	~560	1367722	12545	10 mins	~750
MLP (Figure 7)	~558	1367722	36943	10 mins	not tested
Random Forest	~558	1367722	6400	5 mins	not tested
Ensemble of XGBRegressor	~ 410	320000	~ 1277952000	6 hours	~ 650

- Truncate the training samples to smaller pieces.

4.3 Best-Performing Model

Our best-performing model design is the ensemble of XGBoosting Regressor. Due to the attribute of non-deep learning model, it does not have a training curve or validation curve. However, since we trained 6240 single XGBoosting regressor models, we have validation *RMSE* of 6240 models are in Figure 13.

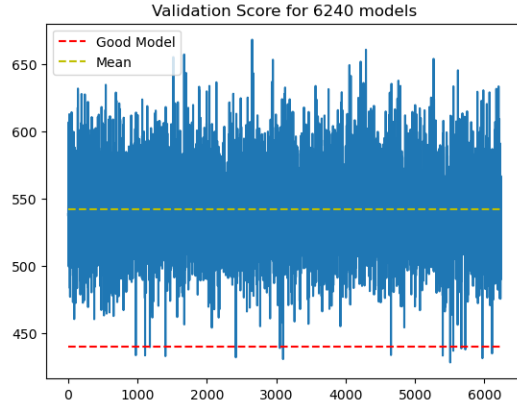


Figure 13: Validation for 6240 models

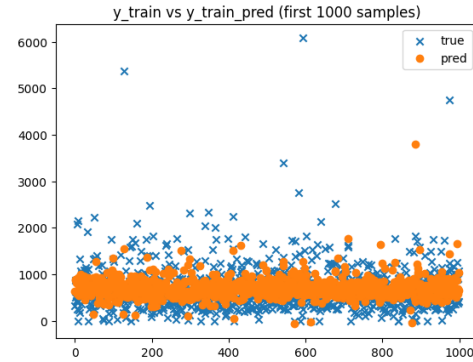


Figure 14: Ground Truth VS Prediction

The ground truth and prediction of 1000 samples in training data are visualized in Figure 14.

After the competition ended, our public score is 650.68864 ranking 11, and private score is 846.48575 ranking 3 on Kaggle. (See Figure 15)

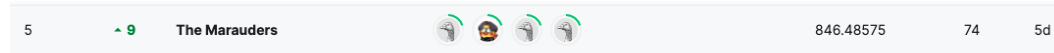


Figure 15: Final Leaderboard

5 Discussion and Future Work

5.1 Result Analysis and Encountered Challenges

We consider data preprocessing, including handling missing values, adding important features, and encoding and normalizing the data, to be the most effective feature engineering strategies. Techniques such as data visualization, model design, and hyper-parameter optimization have proven to be valuable. In particular, data visualization enables us to identify outliers, discover trends, and establish connections between different data types. Selecting the appropriate model and further tuning hyper-parameters are crucial for achieving successful results.

However, we encountered a significant challenge in this project due to the limited availability of information about a specific type of data. Additionally, determining the optimal approach for addressing outliers, which contribute substantially to the final $RMSE$, remains a complex task. This is the reason why the bottleneck is high test error regardless of low validation error.

For aspiring deep learning practitioners tackling similar prediction tasks, we offer the following advice: thoroughly understand the data and potentially useful models before commencing the design process. Gain clarity on the task at hand and identify the most relevant data features. By implementing appropriate data preprocessing techniques, valuable insights can be extracted. Selecting an appropriate model and applying essential techniques such as normalization and optimization can be accomplished through an iterative process of trial and error.

Given the opportunity of additional time and improved hardware resources, we are eager to delve further into the feature engineering process, extracting more valuable information from the original dataset, and experimenting with sophisticated deep learning models.

Our codes are available for review at: <https://github.com/cse151b-sp23/milestone1-the-marauders>

References

Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, pages 785–794, 2016.